

# claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-  
a975fcf895d14d0e4.jsonl`  
- SHA-256: `0a649337e548c0a7b1d62e964acaad19eb9a4a3e3c6061fc7a31976ccb219874`  
- Source modified: `2026-07-06T06:07:56+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_eb99bd9f-440`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

## Transcript

### 1. user (2026-07-06T06:05:57.676Z)

You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-b-domain-schema -> main).  
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral domain schema (Track B). Changed files ONLY: src/tg\_video\_filter/db.py, src/tg\_video\_filter/models.py, tests/test\_db.py, tests/test\_delivery.py. Diff: +1299/-150.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:  
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite; use db.\_\_apply\_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and db.upsert\_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram show main:src/tg\_video\_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest 434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which is NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow trigger), medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

ADVERSARIALLY VERIFY this single finding. Your job is to REFUTE it if you can ? read the exact code at the cited lines, trace the real control flow, and if useful run a repro with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe. Only return CONFIRMED if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. Return REFUTED if the code actually handles it, the trigger cannot occur, or it's pre-existing (not in this diff). Return PLAUSIBLE only if you cannot fully settle it. Also return the corrected severity.

FINDING (dimension shims):

title: Legacy VideoRecord shims cast provider\_source\_id / provider\_message\_id / document\_id with int(), which raises ValueError on any non-Telegram content\_item and breaks the provider-neutrality the PR ships  
severity(claimed): medium  
file:line: src/tg\_video\_filter/db.py:1328

summary: list\_videos\_for\_user (and the get\_video\_by\_\* shims + \_content\_item\_to\_video) unconditionally int()-cast columns that the ADR 0011 schema deliberately makes provider-neutral TEXT, so a single non-Telegram row poisons the whole account read.

failure\_scenario: Seed one account, then create a non-Telegram content\_item via the now-exported provider-neutral API: upsert\_source(Source(provider='whatsapp', provider\_source\_id='group-abc')) + insert\_content\_item(ContentItem(provider='whatsapp', provider\_message\_id='msg-xyz', dedup\_key='wa:msg:group-abc:msg-xyz')). Calling list\_videos\_for\_user(conn, account\_id) then raises `ValueError: invalid literal for int() with base 10: 'group-abc'` at db.py:1328 (int(r['src\_pid'])) and would also fail at :1329 for a non-numeric provider\_message\_id ? crashing the entire /search read for that account rather than returning the Telegram rows. Symmetrically, get\_video\_by\_document\_id/get\_video\_by\_content\_fingerprint (db.py:1148/1173, int(src.provider\_source\_id)) and \_content\_item\_to\_video (db.py:1261-1262) raise if the resolved source/item is non-Telegram, e.g. a content\_item with document\_id='AGADabc' makes get\_video() raise ValueError.

evidence: Repro (venv python) against a fresh :memory: schema: `B list\_videos\_for\_user RAISED: ValueError invalid literal for int() with base 10: 'group-abc'` and `C get\_video RAISED: ValueError invalid literal for int() with base 10: 'AGADabc'`. Cast sites: db.py:1112 (\_source\_to\_channel), 1148 & 1173 (get\_video\_by\_document\_id / get\_video\_by\_content\_fingerprint), 1261-1262 (\_content\_item\_to\_video), 1328-1330 (list\_videos\_for\_user). The columns are TEXT and provider-scoped by design (models.py:136 provider\_source\_id, :165 provider\_message\_id, :172 document\_id; schema db.py:92/106/111). No in-diff writer (insert\_video/upsert\_channel/migration\_003) produces non-numeric values ? all are Telegram-numeric ? so this is latent, not triggerable by shipped callers today; but the PR exports insert\_content\_item/upsert\_source (db.py \_\_all\_\_ :1355-1359) and the schema/docstrings explicitly invite 'whatsapp' rows, so the leak is reachable the moment Track C or any provider-neutral caller writes one.

recommendation: In the legacy shims, gate the int() casts on provider=='telegram' (or catch ValueError / carry the provider through) so a non-Telegram content\_item does not crash account-wide reads. The VideoRecord.channel\_id/message\_id/document\_id fields are int-typed, so the cleaner fix is to have these Telegram-only shims filter to provider='telegram' rows (e.g. add `AND ci.provider='telegram'` to the list\_videos\_for\_user query and short-circuit the get\_video\_by\_\* helpers when the resolved source's provider != 'telegram'), matching their documented 'legacy Telegram view' contract.

## 2. assistant (2026-07-06T06:06:00.211Z)

I'll verify this finding by reading the actual code at the cited lines.

## 3. user (2026-07-06T06:06:01.291Z)

```

1100
1101     async def get_channel(conn: aiomysqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:
1102         """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1103         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1104         return _source_to_channel(src) if src else None
1105
1106
1107     def _source_to_channel(src: Source) -> Channel:
1108         """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`."""
1109         return Channel(
1110             id=src.id,
1111             user_id=src.account_id,
1112             channel_id=int(src.provider_source_id),
1113             title=src.title,
1114             seen_first_at=src.seen_first_at,
1115         )
1116
1117
1118     # --- legacy VideoRecord shims (backwards-compatible view over content_items) -
1119
1120
1121     async def get_video(
1122         conn: aiomysqlite.Connection, user_id: int, channel_id: int, message_id: int
1123     ) -> VideoRecord | None:

```

```

1124     """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).
1125
1126     Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
1127     :class:`Source` + :class:`ContentItem` and projects the latter onto a
1128     :class:`VideoRecord`. New code should call :func:`get_content_item_by_message`.
1129     """
1130     src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1131     if src is None:
1132         return None
1133     item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))
1134     return _content_item_to_video(item, src, user_id, channel_id) if item else None
1135
1136
1137     async def get_video_by_document_id(
1138         conn: aiosqlite.Connection, user_id: int, document_id: int
1139     ) -> VideoRecord | None:
1140         """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1141         dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1142         item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1143         if item is None:
1144             return None
1145         src = await get_source_by_id(conn, item.source_id)
1146         if src is None:
1147             return None
1148         return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1149
1150
1151     async def get_video_by_content_fingerprint(
1152         conn: aiosqlite.Connection,
1153         user_id: int,
1154         size_bytes: int,
1155         duration_s: int,
1156         height: int,
1157         width: int,
1158     ) -> VideoRecord | None:
1159         """Deprecated shim: Tier-2 lookup via ``compute_dedup_key`` + ``content_items``."""
1160         dedup_key = compute_dedup_key(
1161             provider="telegram",
1162             size_bytes=size_bytes,
1163             duration_s=duration_s,
1164             height=height,
1165             width=width,
1166         )
1167         item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1168         if item is None:
1169             return None
1170         src = await get_source_by_id(conn, item.source_id)
1171         if src is None:
1172             return None
1173         return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1174
1175
1176     async def get_source_by_id(conn: aiosqlite.Connection, source_id: int) -> Source | None:
1177         """Resolve a source by its surrogate PK (used by the legacy video shims)."""
1178         conn.row_factory = aiosqlite.Row
1179         async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
1180             row = await cur.fetchone()
1181             return _row_to_source(row) if row else None
1182
1183
1184     async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
1185         """Deprecated shim: insert into ``content_items`` (ADR 0011).
1186
1187         Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1188         if absent), computes the provider-neutral ``dedup_key`` via

```

```

1189 :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1190 ``VideoRecord`` shape is reconstructed from the inserted row.
1191 """
1192 src = await upsert_source(
1193     conn,
1194     Source(
1195         account_id=video.user_id,
1196         provider="telegram",
1197         provider_source_id=str(video.channel_id),
1198         kind="channel",
1199         watched=True,
1200     ),
1201 )
1202 assert src.id is not None # upsert_source populates the surrogate PK
1203 item = ContentItem(
1204     account_id=video.user_id,
1205     source_id=src.id,
1206     provider="telegram",
1207     provider_message_id=str(video.message_id),
1208     content_type="video",
1209     document_id=str(video.document_id) if video.document_id is not None else None,
1210     duration_s=video.duration_s,
1211     width=video.width,
1212     height=video.height,
1213     size_bytes=video.size_bytes,
1214     mime_type=video.mime_type,
1215     dedup_key=compute_dedup_key(
1216         provider="telegram",
1217         document_id=(str(video.document_id) if video.document_id is not None else
1218             None),
1219         source_id=src.id,
1220         provider_message_id=str(video.message_id),
1221         size_bytes=video.size_bytes,
1222         duration_s=video.duration_s,
1223         height=video.height,
1224         width=video.width,
1225     ),
1226     matched=video.matched,
1227     seen_at=video.seen_at,
1228 )
1229 item = await insert_content_item(conn, item)
1230 return _content_item_to_video(item, src, video.user_id, video.channel_id)
1231
1232 async def mark_forwarded(conn: aioredis.Connection, video_id: int) -> None:
1233     """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1234
1235     Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1236     #28). For Phase 1 (before Track C lands) there is no equivalent column on
1237     ``content_items`` ? the ``matched`` flag alone is sufficient because v1
1238     delivery still targets the single ``delivery_target`` and retried delivery
1239     is harmless (forwards are idempotent). This shim is retained so existing
1240     call sites in ``delivery.py`` keep working without a flag day.
1241     """
1242     # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
1243     # The row's ``matched`` flag (set at insert time) is the surviving signal.
1244     return None
1245
1246
1247 def _content_item_to_video(
1248     item: ContentItem, src: Source, user_id: int, channel_id: int
1249 ) -> VideoRecord:
1250     """Project a :class:`ContentItem` onto the legacy :class:`VideoRecord`.
1251
1252     ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;

```

```

1253     per-destination delivery state moves to Track C). Existing callers read it
1254     only to decide whether to skip re-delivery; since forwards are idempotent
1255     the ``False`` default is safe for Phase 1.
1256     """
1257     return VideoRecord(
1258         id=item.id,
1259         user_id=user_id,
1260         channel_id=channel_id,
1261         message_id=int(item.provider_message_id),
1262         document_id=int(item.document_id) if item.document_id is not None else None,
1263         duration_s=item.duration_s,
1264         width=item.width,
1265         height=item.height,
1266         size_bytes=item.size_bytes,
1267         mime_type=item.mime_type,
1268         matched=item.matched,
1269         forwarded=False,
1270         seen_at=item.seen_at,
1271     )
1272
1273
1274 # --- filter config shims (backwards-compatible view over policies) -----
1275
1276
1277 async def load_filter_config(conn: aisqlite.Connection, user_id: int) -> FilterConfig:
1278     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1279
1280     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1281     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1282     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1283     """
1284     policy = await get_default_policy(conn, user_id)
1285     if policy is None:
1286         return FilterConfig()
1287     return FilterConfig.model_validate_json(policy.config_json)
1288
1289
1290 async def save_filter_config(conn: aisqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1291     """Deprecated shim: upsert the account's default :class:`Policy`. """
1292     existing = await get_default_policy(conn, user_id)
1293     await upsert_policy(
1294         conn,
1295         Policy(
1296             id=existing.id if existing is not None else None,
1297             account_id=user_id,
1298             name="default",
1299             config_json=cfg.model_dump_json(),
1300             is_default=True,
1301             created_at=(existing.created_at if existing is not None else _now()),
1302             updated_at=_now(),
1303         ),
1304     )
1305
1306
1307 # --- bulk helpers (used by the manager on startup) -----
1308
1309
1310 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1311     conn: aisqlite.Connection, user_id: int, limit: int = 100
1312 ) -> list[VideoRecord]:
1313     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1314     conn.row_factory = aisqlite.Row
1315     async with conn.execute(
1316         """SELECT ci.*, s.provider_source_id AS src_pid

```

```

1317         FROM content_items ci
1318         JOIN sources s ON s.id = ci.source_id
1319         WHERE ci.account_id = ?
1320         ORDER BY ci.id DESC LIMIT ?""",
1321         (user_id, limit),
1322     ) as cur:
1323         rows = await cur.fetchall()
1324     return [
1325         VideoRecord(
1326             id=r["id"],
1327             user_id=user_id,
1328             channel_id=int(r["src_pid"]),
1329             message_id=int(r["provider_message_id"]),
1330             document_id=(int(r["document_id"]) if r["document_id"] is not None else
None),
1331             duration_s=r["duration_s"],
1332             width=r["width"],
1333             height=r["height"],
1334             size_bytes=r["size_bytes"],
1335             mime_type=r["mime_type"],
1336             matched=bool(r["matched"]),
1337             forwarded=False,
1338             seen_at=r["seen_at"],
1339         )
1340         for r in rows
1341     ]
1342
1343
1344 __all__ = [
1345     "init_db",
1346     "compute_dedup_key",
1347     # Account + session repos (ADR 0010)
1348     "upsert_account",
1349     "get_account",
1350     "get_account_by_telegram_id",
1351     "upsert_telegram_session",
1352     "get_telegram_session",
1353     "list_runnable_sessions",
1354     # Provider-neutral domain repos (ADR 0011)
1355     "upsert_source",
1356     "get_source_by_provider_id",
1357     "get_source_by_id",
1358     "insert_content_item",
1359     "get_content_item_by_message",
1360     "get_content_item_by_dedup_key",
1361     "upsert_policy",
1362     "get_default_policy",
1363     # Legacy User shims (backwards-compatible)
1364     "upsert_user",
1365     "get_user",
1366     "list_enabled_users",
1367     # Legacy Channel / Video / config shims (ADR 0011)
1368     "upsert_channel",
1369     "get_channel",
1370     "get_video",
1371     "get_video_by_document_id",
1372     "get_video_by_content_fingerprint",
1373     "insert_video",
1374     "mark_forwarded",
1375     "load_filter_config",
1376     "save_filter_config",
1377     "list_videos_for_user",
1378 ]
1379

```

#### 4. assistant (2026-07-06T06:06:05.355Z)

Now let me verify the claim by tracing the schema and running a repro. Let me check the models and schema first.

#### 5. user (2026-07-06T06:06:06.118Z)

```
80 );
81
82 -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,
83 -- ``content_items``, and ``policies`` are populated by migration 003 (and by
84 -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
85 -- are created empty here so the schema is complete ? Track C (#28) implements
86 -- the delivery logic that populates them.
87
88 CREATE TABLE IF NOT EXISTS sources (
89     id                INTEGER PRIMARY KEY AUTOINCREMENT,
90     account_id        INTEGER NOT NULL REFERENCES accounts(id),
91     provider          TEXT NOT NULL DEFAULT 'telegram',
92     provider_source_id TEXT NOT NULL,
93     title             TEXT,
94     kind              TEXT NOT NULL DEFAULT 'channel',
95     watched           INTEGER NOT NULL DEFAULT 0,
96     seen_first_at     TEXT NOT NULL,
97     last_seen_at      TEXT,
98     UNIQUE (account_id, provider, provider_source_id)
99 );
100
101 CREATE TABLE IF NOT EXISTS content_items (
102     id                INTEGER PRIMARY KEY AUTOINCREMENT,
103     account_id        INTEGER NOT NULL REFERENCES accounts(id),
104     source_id         INTEGER NOT NULL REFERENCES sources(id),
105     provider          TEXT NOT NULL DEFAULT 'telegram',
106     provider_message_id TEXT NOT NULL,
107     content_type       TEXT NOT NULL DEFAULT 'video',
108     text              TEXT,
109     link_url           TEXT,
110     media_ref         TEXT,
111     document_id       TEXT,
112     duration_s        INTEGER,
113     width             INTEGER,
114     height            INTEGER,
115     size_bytes        INTEGER,
116     mime_type         TEXT,
117     dedup_key         TEXT NOT NULL,
118     matched           INTEGER NOT NULL DEFAULT 0,
119     seen_at           TEXT NOT NULL,
120     UNIQUE (account_id, source_id, provider_message_id),
121     UNIQUE (account_id, dedup_key)
122 );
123
124 CREATE TABLE IF NOT EXISTS policies (
```

#### 6. assistant (2026-07-06T06:06:09.181Z)

The schema confirms these columns are TEXT and provider-neutral. The finding's analysis of the cast sites is accurate. Now the key question for severity: is this triggerable by any shipped caller in THIS PR, or purely latent? Let me run the repro to confirm the crash behavior.

#### 7. user (2026-07-06T06:06:10.234Z)

```
1  ""SQLite persistence layer (aiosqlite).
2
3  Schema, migrations, and repository functions. See ``docs/design.md`` §3 for
```

```

4     the legacy data model and ``docs/adr/0011-provider-neutral-domain-schema.md``
5     for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /
6     ``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
7
8     The repository functions take an ``aiosqlite.Connection`` so tests can inject an
9     in-memory database (``aiosqlite.connect(":memory:")``).
10    """
11
12    from __future__ import annotations
13
14    from collections.abc import Awaitable, Callable, Iterable
15    from datetime import UTC, datetime
16
17    import aiosqlite
18
19    from tg_video_filter.models import (
20        Account,
21        Channel,
22        ContentItem,
23        FilterConfig,
24        Policy,
25        Source,
26        TelegramSession,
27        User,
28        VideoRecord,
29    )
30
31    # --- DDL -----
32
33    _SCHEMA = """
34    CREATE TABLE IF NOT EXISTS accounts (
35        id            INTEGER PRIMARY KEY,
36        telegram_id   INTEGER UNIQUE,
37        provider      TEXT NOT NULL DEFAULT 'telegram',
38        display_name  TEXT,
39        created_at    TEXT NOT NULL
40    );
41
42    CREATE TABLE IF NOT EXISTS telegram_sessions (
43        account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
44        phone         TEXT,
45        session_path  TEXT NOT NULL,

```

## 8. assistant (2026-07-06T06:06:14.030Z)

Let me check the signatures of `upsert\_account`, `upsert\_source`, `insert\_content\_item`, and `\_apply\_schema` to write an accurate repro.

## 9. user (2026-07-06T06:06:15.652Z)

```

191:async def _apply_schema(conn: aiosqlite.Connection) -> None:
254:def compute_dedup_key(
730:async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
898:async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
953:async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) -> ContentItem:

```

## 10. user (2026-07-06T06:06:20.065Z)

```

120    # Option 2A: ``policies`` generalizes ``filter_config``.
121
122
123    class Source(BaseModel):
124        """A configured place we ingest content from (ADR 0011).
125

```

```

126     Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127     providers add their own source kinds). ``provider_source_id`` is the
128     provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129     """
130
131     model_config = ConfigDict(frozen=True)
132
133     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134     account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135     provider: str = Field(default="telegram", description="Provider namespace")
136     provider_source_id: str = Field(
137         description="Provider-native source id (Telegram channel_id, ?)"
138     )
139     title: str | None = None
140     kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141     watched: bool = Field(
142         default=False,
143         description="ADR 0009 selective scanning: ingest from this source iff True",
144     )
145     seen_first_at: datetime = Field(default_factory=_utcnow)
146     last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154         NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155         ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)
160
161         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162         account_id: int = Field(description="FK ? accounts.id")
163         source_id: int = Field(description="FK ? sources.id")
164         provider: str = Field(default="telegram", description="Provider namespace")
165         provider_message_id: str = Field(
166             description="Provider-native message id (Telegram message_id, ?)"
167         )
168         content_type: str = Field(default="video", description="'video' | 'link' |
169 'article'")
170         text: str | None = Field(default=None, description="Caption / post text (nullable in
171 Phase 1)")
172         link_url: str | None = Field(default=None, description="For link/article posts
173 (Phase 1: NULL)")
174         media_ref: str | None = Field(default=None, description="Provider media reference")
175         document_id: str | None = Field(
176             default=None,
177             description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
178         )
179         duration_s: int | None = None
180         width: int | None = None
181         height: int | None = None
182         size_bytes: int | None = None
183         mime_type: str | None = None
184         dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
185 per account")
186         matched: bool = False
187         seen_at: datetime = Field(default_factory=_utcnow)
188
189     class Policy(BaseModel):

```

```
187         """"A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can
```

## 11. assistant (2026-07-06T06:06:22.055Z)

Now let me write the repro to confirm the crash behavior.

## 12. assistant (2026-07-06T06:06:34.364Z)

```
import asyncio
import aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Account, Source, ContentItem

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    acct = await db.upsert_account(conn, Account(id=1, telegram_id=100, display_name="u",
created_at=db._now()))
    uid = acct.id

    # Seed a Telegram row via legacy insert_video path (numeric)
    src_tg = await db.upsert_source(conn, Source(account_id=uid, provider="telegram",
provider_source_id="12345", kind="channel", watched=True))
    item_tg = ContentItem(
        account_id=uid, source_id=src_tg.id, provider="telegram",
        provider_message_id="678", content_type="video", document_id="99",
        dedup_key="tg:doc:99",
    )
    await db.insert_content_item(conn, item_tg)

    # Seed a non-Telegram row via the exported provider-neutral API
    src_wa = await db.upsert_source(conn, Source(account_id=uid, provider="whatsapp",
provider_source_id="group-abc", kind="group", watched=True))
    item_wa = ContentItem(
        account_id=uid, source_id=src_wa.id, provider="whatsapp",
        provider_message_id="msg-xyz", content_type="video", document_id="AGADabc",
        dedup_key="wa:msg:group-abc:msg-xyz",
    )
    await db.insert_content_item(conn, item_wa)

    # A: list_videos_for_user
    try:
        rows = await db.list_videos_for_user(conn, uid)
        print("A list_videos_for_user OK:", [(r.channel_id, r.message_id) for r in rows])
    except Exception as e:
        print(f"A list_videos_for_user RAISED: {type(e).__name__} {e}")

    # B: get_video_by_document_id on the non-TG doc via dedup key -- but note it uses
compute_dedup_key(provider='telegram')
    # Instead test get_video_by_document_id where the resolved source is whatsapp.
    # Craft a telegram-dedup content item whose source is whatsapp is impossible via normal
path.
    # Test the direct claim: a whatsapp source with numeric-looking doc won't resolve via TG
dedup.
    # So test get_video via get_video path with telegram known-good:
    v = await db.get_video(conn, uid, 12345, 678)
    print("B get_video (telegram) OK:", v.channel_id if v else None, v.message_id if v else
None)

    await conn.close()
```

```
asyncio.run(main())
```

### 13. user (2026-07-06T06:06:34.364Z)

<tool\_use\_error>File has not been read yet. Read it first before writing to it.</tool\_use\_error>

### 14. assistant (2026-07-06T06:06:37.785Z)

Let me check `\_now` exists and write to a fresh path.

### 15. user (2026-07-06T06:06:39.517Z)

```
600:def _now() -> datetime:
```

### 16. assistant (2026-07-06T06:06:48.653Z)

```
import asyncio
import aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Account, Source, ContentItem

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    acct = await db.upsert_account(conn, Account(id=1, telegram_id=100, display_name="u",
created_at=db._now()))
    uid = acct.id

    src_tg = await db.upsert_source(conn, Source(account_id=uid, provider="telegram",
provider_source_id="12345", kind="channel", watched=True))
    await db.insert_content_item(conn, ContentItem(
        account_id=uid, source_id=src_tg.id, provider="telegram",
        provider_message_id="678", content_type="video", document_id="99",
        dedup_key="tg:doc:99",
    ))

    src_wa = await db.upsert_source(conn, Source(account_id=uid, provider="whatsapp",
provider_source_id="group-abc", kind="group", watched=True))
    await db.insert_content_item(conn, ContentItem(
        account_id=uid, source_id=src_wa.id, provider="whatsapp",
        provider_message_id="msg-xyz", content_type="video", document_id="AGADabc",
        dedup_key="wa:msg:group-abc:msg-xyz",
    ))

    try:
        rows = await db.list_videos_for_user(conn, uid)
        print("A list_videos_for_user OK:", [(r.channel_id, r.message_id) for r in rows])
    except Exception as e:
        print(f"A list_videos_for_user RAISED: {type(e).__name__}: {e}")

    await conn.close()
```

```
asyncio.run(main())
```

### 17. user (2026-07-06T06:06:48.654Z)

<tool\_use\_error>File has not been read yet. Read it first before writing to it.</tool\_use\_error>

### 18. user (2026-07-06T06:06:54.148Z)

(Bash completed with no output)

### 19. assistant (2026-07-06T06:07:03.186Z)

```
import asyncio
import aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Account, Source, ContentItem

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    acct = await db.upsert_account(conn, Account(id=1, telegram_id=100, display_name="u",
created_at=db._now()))
    uid = acct.id

    src_tg = await db.upsert_source(conn, Source(account_id=uid, provider="telegram",
provider_source_id="12345", kind="channel", watched=True))
    await db.insert_content_item(conn, ContentItem(
        account_id=uid, source_id=src_tg.id, provider="telegram",
        provider_message_id="678", content_type="video", document_id="99",
        dedup_key="tg:doc:99",
    ))

    src_wa = await db.upsert_source(conn, Source(account_id=uid, provider="whatsapp",
provider_source_id="group-abc", kind="group", watched=True))
    await db.insert_content_item(conn, ContentItem(
        account_id=uid, source_id=src_wa.id, provider="whatsapp",
        provider_message_id="msg-xyz", content_type="video", document_id="AGADabc",
        dedup_key="wa:msg:group-abc:msg-xyz",
    ))

    try:
        rows = await db.list_videos_for_user(conn, uid)
        print("A list_videos_for_user OK:", [(r.channel_id, r.message_id) for r in rows])
    except Exception as e:
        print(f"A list_videos_for_user RAISED: {type(e).__name__}: {e}")

    await conn.close()

asyncio.run(main())
```

### 20. user (2026-07-06T06:07:03.220Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro2.py (file state is current in your context ? no need to Read it back)

### 21. user (2026-07-06T06:07:11.508Z)

A list\_videos\_for\_user RAISED: ValueError: invalid literal for int() with base 10: 'group-abc'

### 22. assistant (2026-07-06T06:07:18.143Z)

The repro confirms the crash. `list\_videos\_for\_user` raises `ValueError` when a non-Telegram content\_item exists for the account, poisoning the entire account read rather than returning the Telegram rows.

The code analysis is fully accurate:

- Schema columns are TEXT/provider-neutral (db.py:92, 106, 111).
- The int() casts at db.py:1112, 1148, 1173, 1261-1262, 1328-1330 are unconditional.
- The provider-neutral writers `upsert\_source`/`insert\_content\_item` are exported in `\_\_all\_\_` (db.py:1355-1358).

Now the severity question. Let me verify whether any shipped caller in this PR actually writes a non-Telegram row. All in-diff writers (insert\_video at 1192-1228, upsert\_channel, migration\_003) hardcode `provider="telegram"` and numeric-string ids. Let me confirm no non-Telegram writer ships in the diff and that `list\_videos\_for\_user` isn't called in v1.

### 23. user (2026-07-06T06:07:19.872Z)

No matches found

### 24. user (2026-07-06T06:07:33.647Z)

```
src\tg_video_filter\db.py:565:         AND s.provider = 'telegram'
src\tg_video_filter\db.py:1090:         provider="telegram",
src\tg_video_filter\db.py:1141:     dedup_key = compute_dedup_key(provider="telegram",
document_id=str(document_id))
src\tg_video_filter\db.py:1161:         provider="telegram",
src\tg_video_filter\db.py:1196:         provider="telegram",
src\tg_video_filter\db.py:1206:         provider="telegram",
src\tg_video_filter\db.py:1216:         provider="telegram",
```

### 25. assistant (2026-07-06T06:07:46.372Z)

Confirmed: every writer in `src/` uses `provider="telegram"` with numeric-string ids. No shipped caller in this PR (or anywhere in `src/`) creates a non-Telegram content\_item, and `list\_videos\_for\_user`/`get\_video\_by\_\*` shims have no non-test callers writing non-numeric values. So the defect is genuinely latent ? not triggerable by any shipped caller today.

The finding's own evidence acknowledges this: "this is latent, not triggerable by shipped callers today." The trigger requires a future Track C caller (or a test) to use the now-exported provider-neutral API with a non-Telegram provider.

Assessment:

- The defect genuinely ships in this PR's diff (the int() casts are all in changed code, and the provider-neutral writers are newly exported).
- The described wrong behavior (account-wide read crash) is real and reproducible.
- But the trigger cannot occur via any shipped caller today ? it needs a future non-Telegram writer that doesn't exist yet in the codebase.

The finding claims severity "medium." Per the rubric, a latent defect whose trigger is not reachable by any current caller is a "medium (latent/edge)" at most, and arguably "low" given the trigger requires future code that doesn't exist. However, it's a real correctness landmine in code the PR ships and explicitly invites (docstrings/schema mention 'whatsapp'), so it's not pure hygiene. Medium is defensible but I'd lean it toward low given zero reachability today. I'll confirm the defect but correct severity down to low, since no shipped path triggers it and the whole premise is "reachable the moment future Track C writes a non-Telegram row."

Actually, the casts genuinely ship and would break the moment provider-neutrality is exercised, which is the PR's stated purpose. The int() casts silently contradict the ADR-0011 contract. That's a real latent bug ? medium is the honest floor between "latent/edge" (medium) and "hygiene" (low). Given it requires non-existent future code to trigger, low is more honest.

### 26. user (2026-07-06T06:07:56.612Z)

Structured output provided successfully